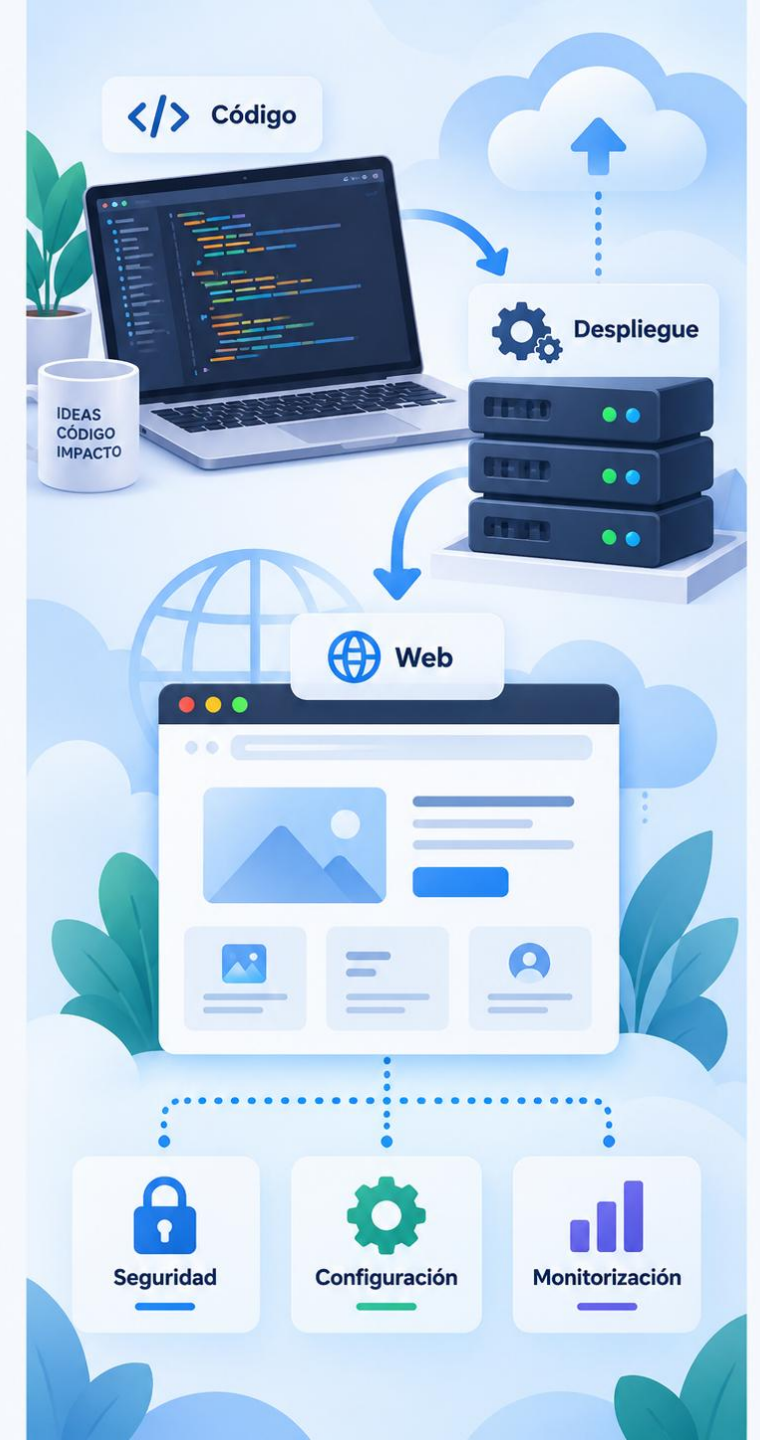


Arquitecturas web y proceso de despliegue

Tema1 — Punto de partida



Objetivo de la sesión

Cambiar la pregunta:

de “¿funciona?” a “¿se puede desplegar?”

Funciona en mi equipo

- Dependencias locales
- Datos y rutas personales
- Secretos mal ubicados
- Pasos recordados de memoria



Funciona para otros

- Proceso repetible
- Configuración externa
- Evidencias y diagnóstico
- Actualizaciones controladas

Idea clave: desplegar es hacer operativo un sistema para usuarios reales.

Tecnologías cambian; buenas prácticas permanecen



Qué buscamos en un despliegue

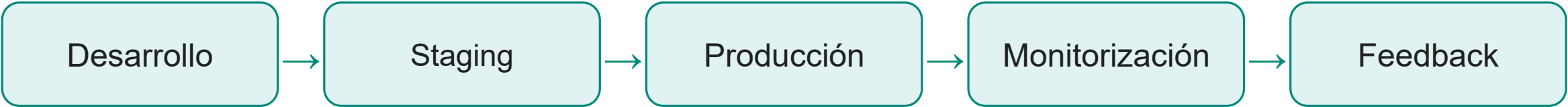
Siete propiedades que iremos construyendo poco a poco

Propiedad	Significado
Reproducible	mismo procedimiento, resultado equivalente
Trazable	saber qué versión está funcionando
Configurable	mismo artefacto, distinta configuración
Seguro	secretos fuera del código
Observable	estado, logs y métricas
Escalable	capacidad ante carga
Actualizable	cambiar y volver atrás

No todo se consigue el primer día: el módulo irá incorporando estas propiedades por capas.

Entornos: desarrollo, staging y producción

El artefacto debería mantenerse; la configuración cambia



Entorno	Para qué sirve	Qué cambia
Desarrollo	trabajar rápido	datos de prueba, configuración local
Staging	validar antes de publicar	configuración parecida a producción
Producción	usuarios reales	datos reales, seguridad y disponibilidad

Regla práctica: no recompiles la aplicación solo para cambiar una contraseña o una URL.

HTTP: el idioma del diagnóstico

No es solo para programar APIs

Petición

```
GET /productos HTTP/1.1  
Host: ejemplo.test  
Accept: application/json
```



Respuesta

```
HTTP/1.1 200 OK  
Content-Type: application/json  
Cache-Control: no-cache
```

Familia	Pista inicial
2xx	atendido correctamente
3xx	redirección o nuevo destino
4xx	problema con petición, recurso o acceso
5xx	servidor o componente intermedio falla

Un código HTTP da una pista; no siempre dice la causa exacta.

La diferencia no es si cambia, sino si hay que ejecutar lógica

Contenido estático

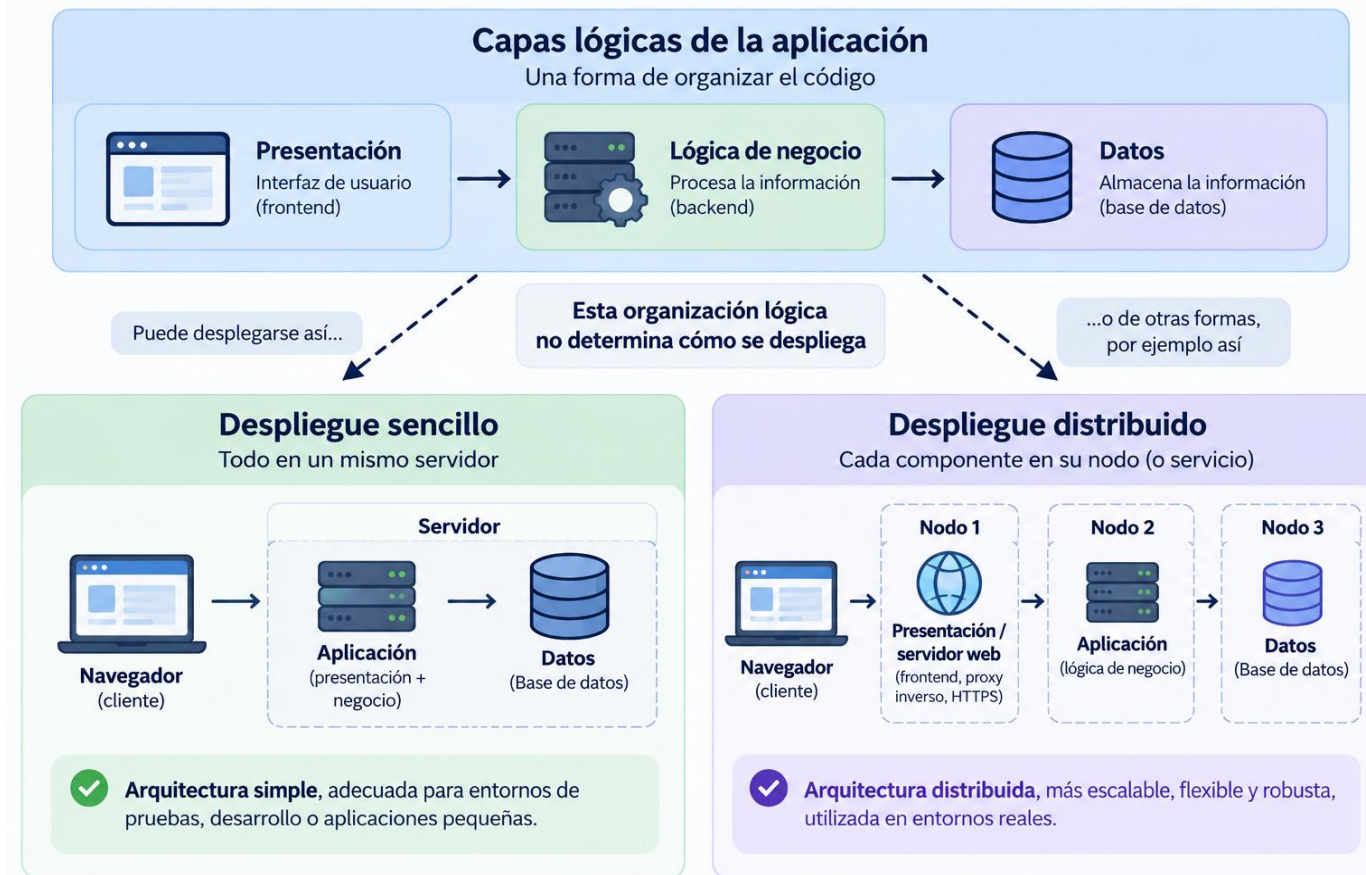
- HTML, CSS, JS, imágenes
- El fichero ya existe
- Un servidor web puede leerlo y entregarlo
- Buen candidato a caché o CDN

Contenido dinámico

- Se genera al recibir la petición
- Ejecuta lógica de aplicación
- Puede consultar bases de datos
- Depende del estado del sistema

Capas lógicas ≠ unidades de despliegue

Organizar código no decide automáticamente dónde se ejecuta



Pregunta clave: ¿cuántos servidores hacen falta para una aplicación con tres capas? Respuesta: depende.

Monolito no significa antiguo

Microservicios no significan automáticamente mejor

Monolito modular

- Una unidad desplegable
- Menos complejidad operativa
- Encaja bien en equipos pequeños/medios
- Puede organizarse internamente por módulos

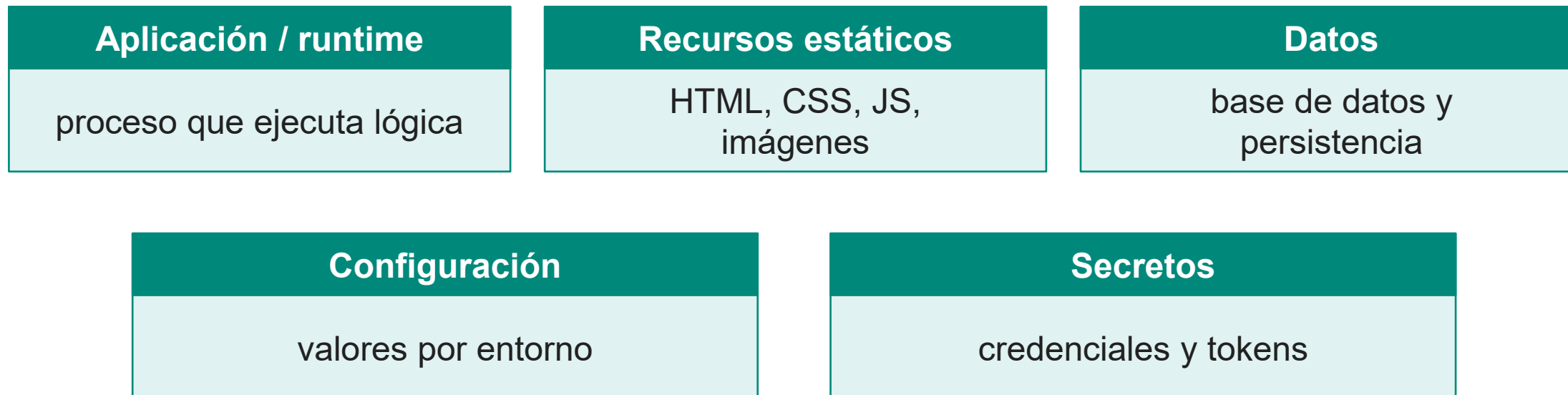
Microservicios

- Varias unidades desplegables
- Escalado y ciclos independientes
- Más red, logs, datos y coordinación
- Solo compensa si el problema lo justifica

Regla: la complejidad debe ser proporcional al problema.

Piezas básicas de un despliegue

Lo que debe existir además del código



Una arquitectura puede ser sencilla, pero estas responsabilidades no desaparecen.

Escaparate: el hilo conductor

El problema interesante no será programar la app, sino desplegarla

Primera versión	Más adelante
• Frontend servido desde Spring Boot • API y frontend en el mismo contenedor y puerto • Backend empaquetado como WAR ejecutable • PostgreSQL como base de datos • Configuración recibida desde fuera	• Separaremos piezas • Añadiremos Nginx • Mediremos y observaremos • Automatizaremos con CI/CD • Orquestaremos despliegues

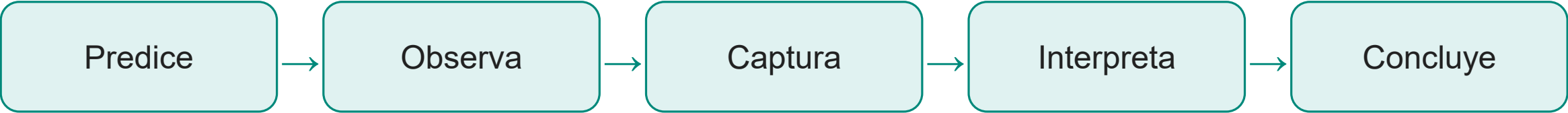
Separar frontend no convierte automáticamente el sistema en microservicios.

Desplegar es un recorrido, no un salto



Actividad 1.1: Qué se puede saber de un despliegue desde fuera

Evidencia, inferencia y límites



Categoría	Significado
Observado	hay evidencia directa
Inferido	hay indicios, pero no demostración completa
No observable	no se puede conocer desde fuera con esas herramientas

Herramientas iniciales: navegador, DevTools y curl.

Lo mínimo que debe quedar claro

- ✓ Una aplicación que solo funciona en tu equipo todavía no está desplegada.
- ✓ El despliegue moderno busca reproducibilidad, trazabilidad y automatización progresiva.
- ✓ HTTP proporciona pistas útiles para diagnosticar.
- ✓ Capas lógicas y unidades de despliegue responden a preguntas distintas.
- ✓ Monolito puede ser una solución actual si está bien organizado.
- ✓ Observar no es lo mismo que inferir.

Próxima sesión: el repositorio se convierte en la fuente de verdad del despliegue.